

Parallélisme - Rapport exercice II

1. Présentation de l'exercice

Dans cette exercice j'ai du programmer en C et avec l'aide OpenMp un programme additionnant tous les éléments d'une matrice $N \times N$. Le but était surtout de comparer les différentes stratégies de parallélisation du programme.

J'ai donc opter pour une simple Matrice remplie de 1 (*i.e* $(J_N)_{i,j} = 1 : \forall i, j \in \llbracket 1, N \rrbracket$).

2. Calcule du temps

Afin d'éviter les incohérence dû à la mise en cache, j'ai créer un script python [Chapitre 4.3] qui exécuté le programme avec des arguments d'exécutions différents à chaque fois.

3. Explications

3.1. Approche naïve

Pour commencer j'ai implémenter un algorithme séquentielle, avec seulement un parcours de la matrice.

```
///@returns renvoie le temps pris par le calcule
int addSequential(const int *m, unsigned int size, float *time) {
    double start, stop;
    int result = 0;

    start = omp_get_wtime();
    for (unsigned int i = 0; i < size * size; i++)
        result += m[i];
    stop = omp_get_wtime();
    *time = stop - start;
    return result;
}
```

Le temps pris sur la machine [Chapitre 4.1] est de $\sim 0.86s$ sur une taille de 30000.

3.2. Parallélisé, somme partielle simple

Pour cette première implémentation, j'ai utiliser un tableau contenant les sommes partielles, qui sont ensuite réduites sur un seul thread grâce à la directive `#pragma omp single`.

```
///@returns renvoie le temps pris par le calcule
int addParaSimplePartialSum(const int *m, unsigned int size, unsigned int nbT,
                           float *time) {
    double start, stop;
    int result = 0;

    // Tableau des sommes partielles aloué dans la heap.
    int *partial_sums = calloc(sizeof(int), nbT);
    start = omp_get_wtime();

    // sur nbT threads
#pragma omp parallel num_threads(nbT)
    {
        unsigned int thread_nb = omp_get_thread_num();
#pragma omp for
        for (unsigned int i = 0; i < size * size; i++)
```

```

    partial_sums[thread_nb] += m[i];

    // réduction des sommes partielles
#pragma omp single
    for (unsigned int i = 0; i < nbT; i++)
        result += partial_sums[i];
}
stop = omp_get_wtime();
*time = stop - start;

free(partial_sums);

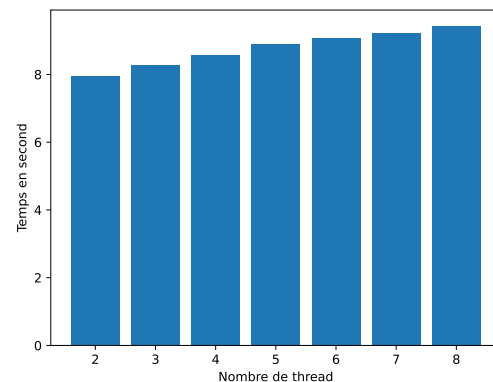
return result;
}

```

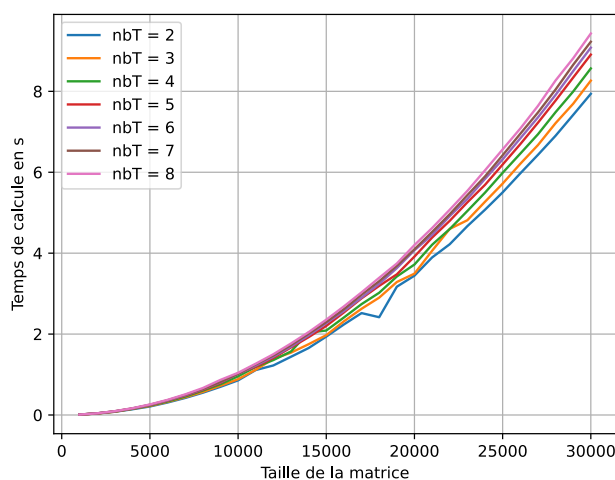
Voici une synthèse des temps en fonction du nombre de thread pour une taille de 30000 :

Temps pris par l'implémentation "somme partielle simple" en fonction du nombre de thread pour 30000 éléments.

Nombre de thread	Temps pris
2	7.93929s
3	8.265104s
4	8.566185s
5	8.907769s
6	9.080031s
7	9.226128s
8	9.429946s



Temps pris par l'implémentation (nbT : nombre de thread)



On peut constater une augmentation du temps en fonction du nombre de thread, cela est probablement dû au nombre de thread attendant pour la réduction, avec le bloc single. De plus la complexité en temps semble quadratique pour tout les nombres de thread.

3.3. Parallélisé, réduction atomique

Pour cette approche j'ai utilisé la directive `#pragma omp atomic` afin de résoudre la somme.

```
///@returns renvoie le temps pris par le calcul
int addParaAtomicPartialSum(const int *m, unsigned int size, unsigned int nbT,
                           float *time) {
    double start, stop;
    int result = 0;

    start = omp_get_wtime();

#pragma omp parallel num_threads(nbT)
    {
        int partial_sum = 0;
#pragma omp for
        for (unsigned int i = 0; i < size * size; i++)
            partial_sum += m[i];

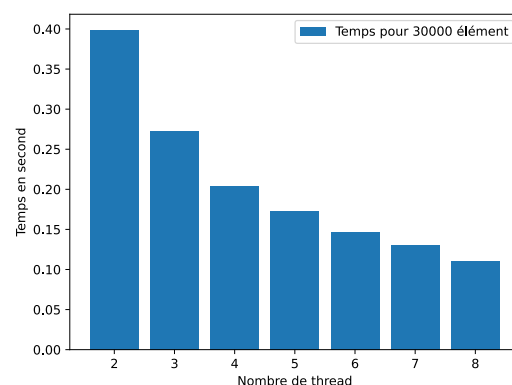
        // réduction des sommes partielles
#pragma omp atomic
        result += partial_sum;
    }
    stop = omp_get_wtime();
    *time = stop - start;

    return result;
}
```

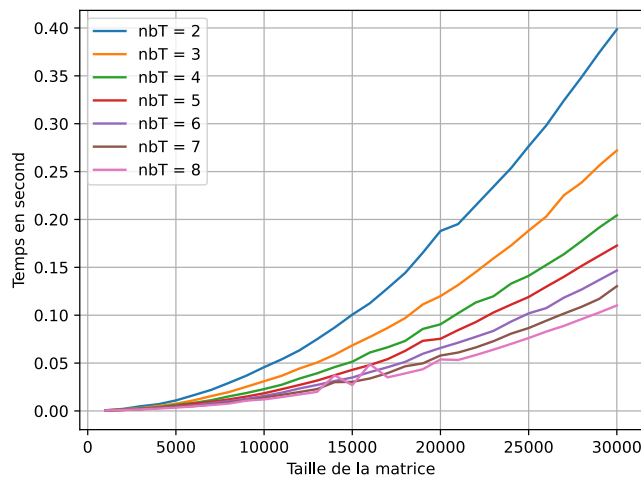
Voici une synthèse des temps en fonction du nombre de thread pour une taille de 30000 :

Temps pris par l'implémentation "réduction atomique" en fonction du nombre de thread pour 30000 éléments.

Nombre de thread	Temps pris
2	0.398453s
3	0.272031s
4	0.204225s
5	0.172668s
6	0.14665s
7	0.130281s
8	0.110147s



Temps pris par l'implémentation (nbT : nombre de thread)



On peut constater une nette amélioration par rapport à l'implémentation précédentes, et de plus le temps s'améliore en fonction du nombre de thread, les threads n'ont pas à s'attendre les un les autre afin d'incrémenter le résultat.

La complexité semble rester quadratique.

3.4. Parallélisé, réduction optimisé

Pour cette dernière implémentation nous avons utiliser la réduction directement fourni par OpenMp : `reduction(+ : result)`, qui est donc plus optimiser qu'une « écrite à la main ».

```
///@returns renvoie le temps pris par le calcul
int addParaOptimizedPartialSum(const int *m, unsigned int size,
                               unsigned int nbT, float *time) {

    double start, stop;
    int result = 0;

    start = omp_get_wtime();

    // réduction du résultat en somme sur result.
    #pragma omp parallel num_threads(nbT) reduction(+ : result)
    {
        #pragma omp for
        for (unsigned int i = 0; i < size * size; i++)
            result += m[i];
    }
    stop = omp_get_wtime();
    *time = stop - start;
    return result;
}
```

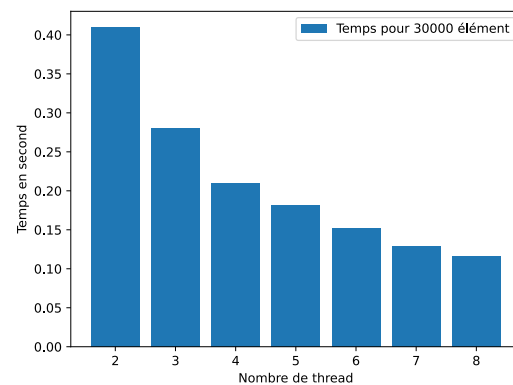
Voici une synthèse des temps en fonction du nombre de thread pour une taille de 30000 :

Temps pris par l'implémentation "réduction optimiser" en fonction du nombre de thread pour 30000 éléments.

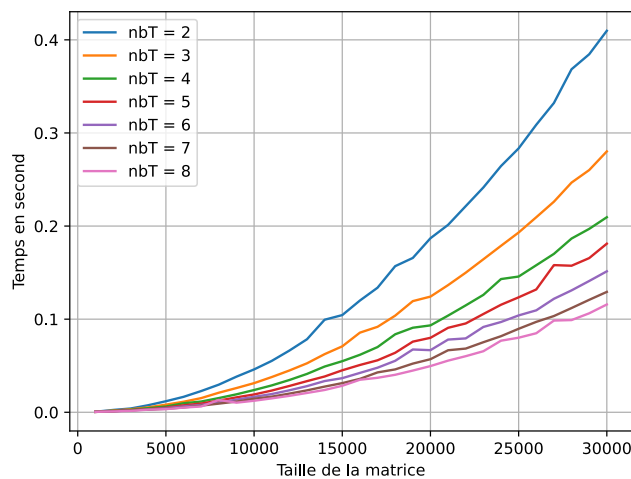
Nombre de thread	Temps pris
2	0.409742s
3	0.280176s

Nombre de thread	Temps pris
4	0.209482s
5	0.181079s

Nombre de thread	Temps pris
6	0.151386s
7	0.129296s
8	0.115773s



Temps pris par l'implémentation (nbT : nombre de thread)



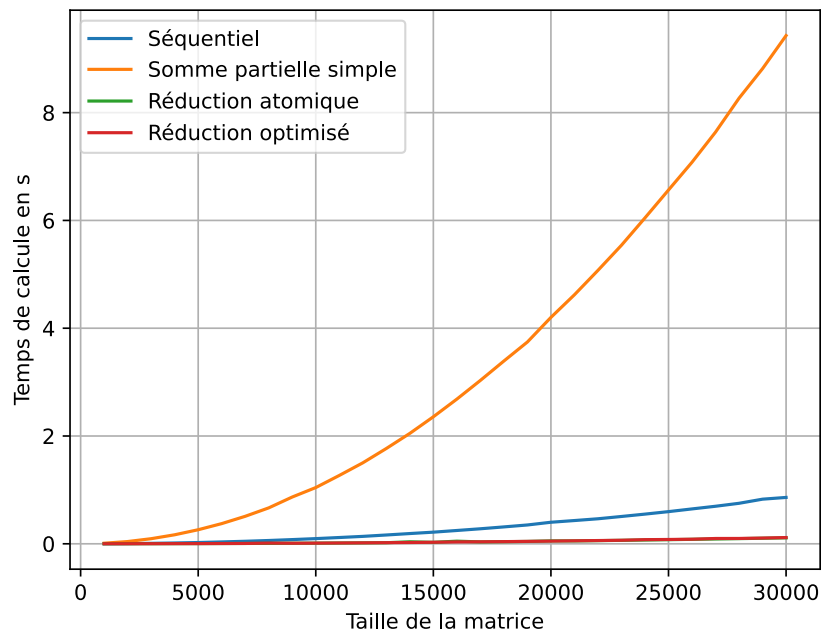
On obtient un résultat proche de l'approche précédente, l'implémentation optimiser d'OpenMp doit ressembler à celle avec les atomiques.

3.5. Quelques graphes de comparaison

Toutes les mesures on était synthétiser dans un document Json, en voici des comparaison graphiques.

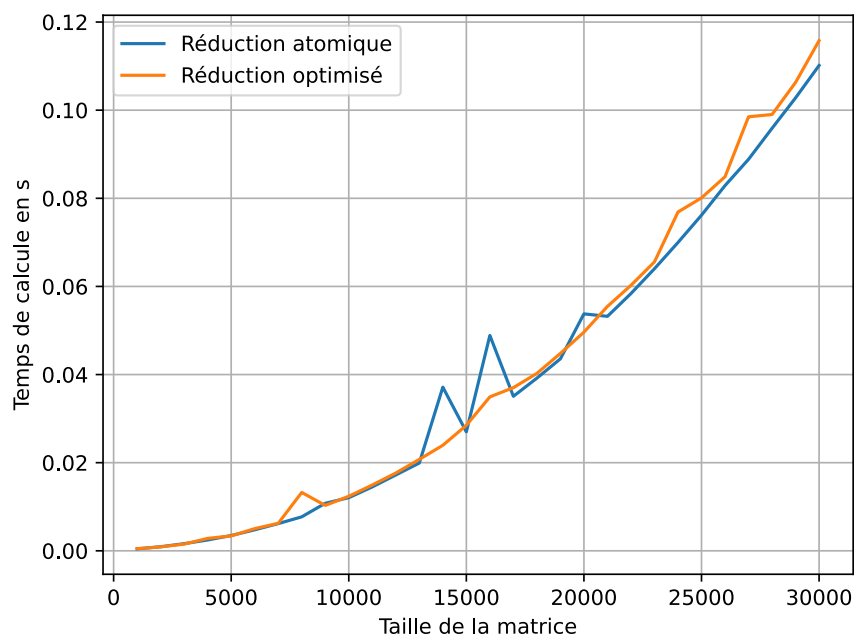
3.5.1. Temps en fonction du nombre d'élément pour chacune des approche :

(Le nombre de thread pour est fixé à 8)



3.6. Temps en fonction du nombre d'élément pour les réduction atomique et optimisé :

(Le nombre de thread pour est fixé à 8)



On voit bien que les deux méthodes sont très proches en termes de performance, l'atomique est plus « bruité » dans ses résultats.

4. Annexe

4.1. Machine utilisé

Tous les temps ont été enregistrés sur un ordinateur portable avec un processeur x86-64 de huit cœurs physiques dont deux virtuels par cœur, cadencé à ~3.5GHz (sur secteur).

4.2. Données récolter

Toutes les mesures effectuées sont stockées dans un document *json* récupérées grâce à la sortie standard du programme, et a un script python, trouvable sur le [GitHub](#).

4.3. Code

L'entièreté du code se trouve sur le [GitHub](#)